Software Design Specification

for

Eco Volunteer Match

Prepared by:

P. Harika: Team Lead and User Authentication & Profiles
D. Manasvini: Location-Based Event Matching
P. Srija: Event Creation & Management.
P. Ananya: Reminder and Notification Engineer
S. Sneha: Testing & Deployment Engineer
J. Akshaya: Backend & Documentation
Venkat: Rewards & Gamification
Akarshan: Testing, Quality Assurance & Documentation

Document Information

Title: Eco Volunteer Match

Document Version No: 1.0

Project Manager:
Team 16

Document Version Date:3/04/26

Prepared By:
P. Harika, D. Manasvini, S. Sneha, P. Srija, P. Ananya, Akshaya, Venkat, Akarshan

Preparation Date:3/04/26

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename

Table of Contents

1 INTRODUCTION 4

1.1 PURPOSE 4
1.2 SCOPE 4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 4
1.4 REFERENCES 4

2 USE CASE VIEW 4

2.1 USE CASE 4

3 DESIGN OVERVIEW 4

3.1 DESIGN GOALS AND CONSTRAINTS 5
3.2 DESIGN ASSUMPTIONS 5
3.3 SIGNIFICANT DESIGN PACKAGES 5
3.4 DEPENDENT EXTERNAL INTERFACES 5
3.5 IMPLEMENTED APPLICATION EXTERNAL INTERFACES 5

4 LOGICAL VIEW 5

4.1 DESIGN MODEL 6
4.2 USE CASE REALIZATION 6

5 DATA VIEW 6

5.1 DOMAIN MODEL 6
5.2 D ATA MODEL (PERSISTENT DATA VIEW) 6
5.2.1 Data Dictionary 6

6 EXCEPTION HANDLING 6

7 CONFIGURABLE PARAMETERS 6

8 QUALITY OF SERVICE 7

8.1 AVAILABILITY 7
8.2 SECURITY AND AUTHORIZATION 7
8.3 LOAD AND PERFORMANCE IMPLICATIONS 7
8.4 MONITORING AND CONTROL 7

1 Introduction

The Software Design Specification (SDS) explains the design of the Eco Volunteer Match Platform. It provides an overview of how the system works, what features it has, and how its different parts are connected. This document is mainly useful for understanding the application's structure and development.

1.1 Purpose

The purpose of this document is to describe the design and architecture of the Eco-Volunteer Match Platform. It explains how the system is structured, how modules interact, and how features are implemented.
This document helps developers, evaluators, and stakeholders understand the internal working of the system.

1.2 Scope

The Eco-Volunteer Match Platform is a full-stack application that connects volunteers with environmental activities such as:

- Tree plantation drives
- Clean-up campaigns
- Awareness programs etc

The system supports:

- Event discovery
- Event registration
- Location-based matching
- Impact tracking
- Notifications

1.3 Definitions, Acronyms, and Abbreviations

SDS - Software Design Specification
API - Application Programming Interface
DB - Database
UI - User Interface
GPS - Location tracking system
FCM - Firebase Cloud Messaging
JWT - JSON Web Token

1.4 References

- IEEE guidelines for Software Design
- Class notes and materials
- Online sources used for understanding system design

2 Use Case View

This section explains how users will interact with the app and what actions they can perform. It includes the main features that users will use in the system.

2.1 Use Case

User Registration

- User enters name, email, password
- System validates input
- Account is created and stored in database

User Login

- User enters username and password
- System checks the credentials
- If correct, User gains access

Event Discovery

- User opens app
- System shows nearby events using maps
- Events are filtered based on location

Event Registration

- User selects event
- Clicks “Register”
- System stores registration

Event Creation (Organizer)

- Organizer inputs event details
- Event is stored
- Admin approval required

Impact Tracking

- System calculates:
    - Number of events
    - Volunteer hours
- Dashboard displays results

Notification System

- System sends:
    - Event reminders
    - Registration confirmation

3 Design Overview

This section gives an overview of the overall design of the Eco Volunteer Match Platform. It shows how the system is organised at a high level and how different parts are connected. The design follows the system requirements and includes major components like user interface, authentication, reminders, notifications, and emergency features. It also shows how these modules are divided and work together to form the complete system.

3.1 Design Goals and Constraints

Goals

- Easy-to-use interface
- Scalable system
- Secure authentication
- Real-time updates

Constraints

- Limited development time
- Mobile performance constraints

3.2 Design Assumptions

- Users have smartphones
- Internet connection available
- Users provide correct data
- External APIs are available

3.3 Significant Design Packages

System is divided into modules:

- User Management Module
- Event Management Module
- Location Module
- Notification Module
- Impact Module
- Admin Module

3.4 Dependent External Interfaces

External System	Module	Purpose
Firebase Auth	Authentication	User login
Firebase FCM	Notification	Push notifications
React (Leaflet)	Location	Event discovery
MongoDB Atlas	Database	Data storage

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces that this design makes available for other applications.

Interface	Module	Function
User API	Backend	Manage users

Interface	Module	Function
Event API	Backend	Manage events
Notification API	Backend	Send alerts
Impact API	Backend	Track analytics

4 Logical View

The logical view describes the internal structure of the system, including how different layers, modules, and classes interact to fulfil system functionalities.
The Eco-Volunteer Match Platform follows a layered architecture, ensuring separation of concerns, scalability, and maintainability.

4.0 System Layers

1. Presentation Layer (Frontend Layer)
This is the **user-facing layer** where users interact with the system through a mobile application.
Technology Used:
- React Native (Expo)

Responsibilities:
- Display UI screens (Login, Events, Map, Dashboard)
- Collect user input (forms, clicks, navigation)
- Send requests to backend APIs
- Display responses *received from backend*

Key Components:
- Login/Register Screen
- Event Discovery Screen
- Map View (location-based events)
- Profile Dashboard
- Impact Dashboard

2. Application Layer (Backend API Layer)
This layer acts as the **core processing unit** of the system.
Technology Used:
- Node.js
- Express.js

Responsibilities:
- Process client requests
- Apply business logic
- Validate input data
- Communicate with database

Example:
When a user registers for an event:

1. Request sent from frontend
2. Backend validates request
3. Stores data in database
4. Sends confirmation response

3. Service Layer (Business Logic Layer)
This layer contains **core functionalities and system rules**.
Responsibilities:

- Event matching logic (location-based filtering)
- Notification triggering
- Impact calculation
- Role-based access control

Example Logic:

- Calculate distance between user and event
- Send reminder before event starts
- Update volunteer hours after participation

4. Database Layer (Data Storage Layer)
This layer manages **persistent data storage**.
Technology Used:

- MongoDB Atlas

Responsibilities:

- Store user data
- Store event data
- Store registrations
- Store impact metrics

Advantages:

- Scalable
- Flexible schema
- Cloud-base

4.1 Design Model

This section defines the **core classes and their responsibilities**, similar to a class diagram explanation.

1. User Class

Attributes:

- user_id
- name
- email
- password
- role (Volunteer / Organizer / Admin)
- location

Methods:

- registerUser()
- loginUser()
- updateProfile()
- viewDashboard()

Responsibility:

Manages all user-related operations including authentication and profile handling.

2. Event Class

Attributes:

- event_id
- title
- description
- location
- date
- organizer_id

Methods:

- createEvent()
- updateEvent()
- deleteEvent()
- getNearbyEvents()

Responsibility:

Handles event creation, management, and retrieval.

3. Registration Class

Attributes:

- registration_id
- user_id
- event_id
- status

Methods:

- registerForEvent()
- cancelRegistration()

Responsibility:

Links users with events and tracks participation.

4. Impact Class

Attributes:

- user_id
- total_events
- volunteer_hours

- badges

Methods:

- updateImpact()
- calculateHours()
- generateStats()

Responsibility:

Tracks and calculates user contributions.

5. Notification Class

Attributes:

- notification_id
- user_id
- message
- timestamp

Methods:

- sendNotification()
- scheduleReminder()

Responsibility:

Handles alerts and reminders using Firebase Cloud Messaging.

4.2 Use Case Realisation

Use Case 1: User Login

Step-by-Step Flow:

1. User enters email and password
2. Frontend sends request to backend
3. Backend forwards to Firebase Authentication
4. Firebase validates credentials
5. If valid → token generated
6. User is redirected to dashboard
7. If invalid → error message displayed

Use Case 2: Event Registration

Step-by-Step Flow:

1. User selects an event
2. Clicks “Register”
3. Request sent to backend
4. Backend checks:
    - User authentication
    - Event availability
5. Registration stored in database
6. Confirmation notification sent

Use Case 3: Location-Based Event Discovery

Step-by-Step Flow:

1. User opens map screen
2. App fetches user location (GPS)

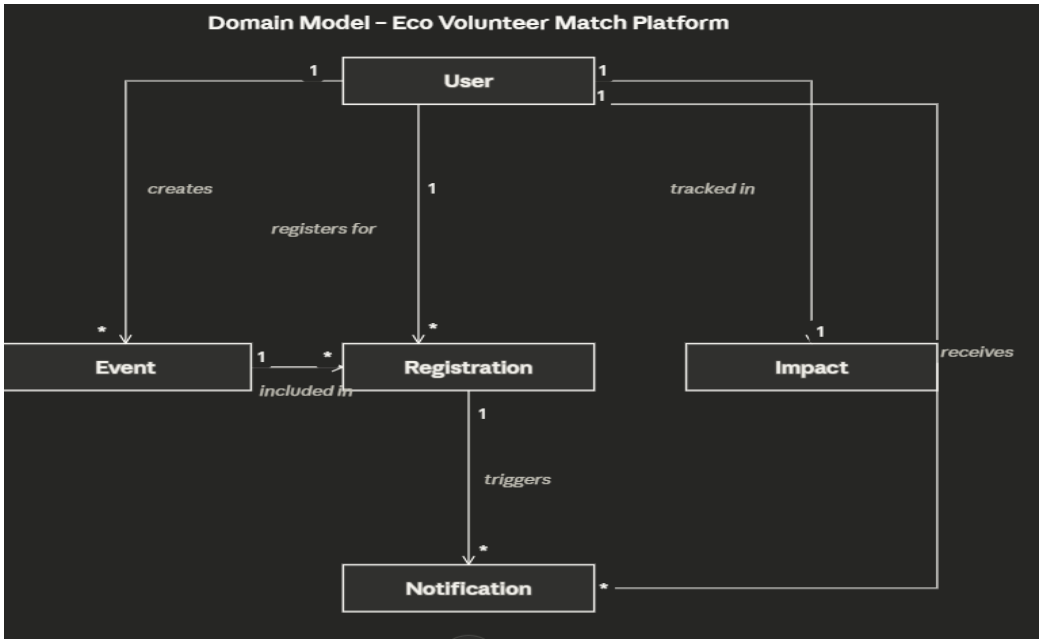
3. Backend queries MongoDB using geo-queries
4. Nearby events returned
5. Events displayed on map

Use Case 4: Impact Tracking
Step-by-Step Flow:
1. After event completion
2. System updates participation record
3. Volunteer hours calculated
4. Dashboard updated
5. Badges awarded if criteria met

5 Data View

This section describes how data is stored and managed in the system. It explains the structure of data and how different data elements are related.

5.1 Domain Model



5.2 Data Model (persistent data view)

The data model below shows how collections in MongoDB relate to each other. Since MongoDB is a NoSQL database, relationships are maintained through reference fields (foreign key equivalents).
Entity Relationship Overview:
- A User can register for many Events (through the Registration collection)
- An Event belongs to one Organizer (User with organizer role)
- A Registration links one User to one Event
- An Impact record belongs to one User and aggregates their activity
- A Notification is sent to one User and relates to one Event

5.2.1 Data Dictionary

Users Collection:

Field	Type	Description
user_id	String	Unique identifier
name	String	User name
email	String	Login email
password	String	Encrypted password
role	String	User role
location	GeoJSON	User location

Events Collection:

Field	Type	Description
event_id	String	Unique ID
title	String	Event name
description	String	Details
location	GeoJSON	Coordinates
date	Date	Event date
organizer_id	String	Creator

Registrations Collections:

Field	Type	Description
registration_id	String	Unique ID
user_id	String	Volunteer
event_id	String	Event
status	String	Registered/Cancelled

Impact Collection:

Field	Type	Description
user_id	String	Reference
events_joined	Number	Count
volunteer_hours	Number	Hours
badges	Array	Earned badges

6 Exception Handling

1. Authentication Errors

- Invalid credentials - “Incorrect email/password”
- Token expired - forced logout

2. Input Validation Errors

- Missing fields - prompt user
- Invalid format - validation message

3. Event Errors

- Event full - registration denied

- Event not found - error response

4. Network Errors

- Timeout - user notification

5. Notification Errors

- Failed push notification - retry queue

7 Configurable Parameters

This table describes the simple configurable parameters (name / value pairs).

Parameter	Description	Dynamic
Notification Settings	Enable/disable alerts	Yes
Location Radius	Distance filter (e.g., 5km)	Yes
Event Preferences	Filter by category	Yes
Language Settings	UI language	Yes

8 Quality of Service

This section describes how the system maintains availability, security, performance, and monitoring during its usage.

8.1 Availability

- System hosted on cloud → high uptime
- Redundant services ensure reliability
- Maintenance handled with minimal disruption

8.2 Security and Authorization

- Firebase Authentication for secure login
- JWT tokens for session management
- Role-based access control
- Encrypted data storage

8.3 Load and Performance

- Optimized database queries
- Scalable backend architecture
- Supports concurrent users

8.4 Monitoring and Control

- Logging system for errors
- Performance monitoring tools
- Admin dashboard for tracking system usage
- Alerts for system failures